

รายงานผลการปฏิบัติการ ใบงานที่ 1

การพัฒนา Desktop Application 'Media Player PRO' ด้วย AI Agent

(Vibe Coding ด้วย Python, PyQt6, QtMultimedia และ PyInstaller)

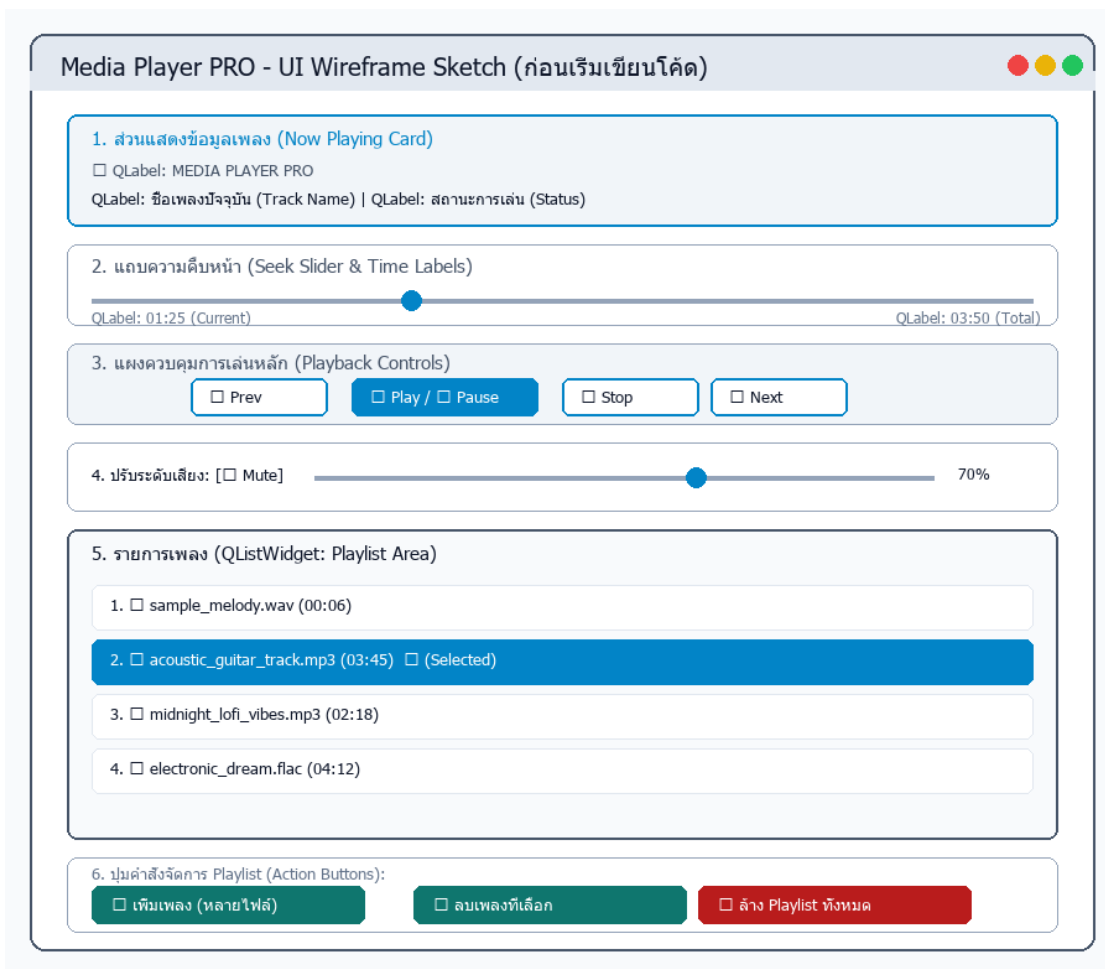
1. การกำหนด Requirement และเกณฑ์การยอมรับ

ตามข้อกำหนดของรายวิชา ผู้เรียนต้องเริ่มต้นจากการกำหนดความต้องการและขอบเขตของแอปพลิเคชันอย่างชัดเจน โดยไม่ปล่อยให้ AI ตีความฟังก์ชันเองตามอำเภอใจ ซึ่งแอปพลิเคชัน Media Player PRO มีข้อกำหนดดังนี้:

- **Playlist หลายไฟล์:** ผู้ใช้สามารถกดเลือกไฟล์เสียงได้พร้อมกันหลายไฟล์ (Multi-selection) เข้าสู่ Playlist
- **การเล่นเพลงจาก Playlist:** ผู้ใช้สามารถคลิกเลือกเพลงใน Playlist แล้วกด Play หรือดับเบิลคลิกเพื่อเริ่มเล่นได้ทันที
- **การจัดการ Playlist:** สามารถเลือกลบเพลงเฉพาะรายการที่ต้องการ (Remove Selected) หรือสั่งล้างรายการเพลงทั้งหมด (Clear Playlist) ได้
- **แถบความคืบหน้า (Seek Slider):** มี Slider แสดงเวลาที่เล่นไปแล้ว และระยะเวลาเต็มของเพลง
รองรับการคลิกหรือลากเพื่อกระโดดข้ามเวลา (Seek)
- **การปรับระดับเสียง (Volume 0-100%):** ปรับเสียงได้ละเอียดตั้งแต่ 0 ถึง 100% พร้อมปุ่ม Mute/Unmute
ที่จดจำระดับเสียงเดิม
- **ความเสถียรและความปลอดภัย (Fallback):** หากกดเล่นขณะที่ยังไม่มีเพลง หรือไฟล์เพลงสูญหาย
ระบบต้องแจ้งเตือนผ่านกล่องข้อความ (QMessageBox) โดยโปรแกรมต้องไม่ Crash
- **การสร้างตัวติดตั้ง Release (.exe):** สามารถคอมไพล์เป็นไฟล์ .exe พร้อมไอคอนโปรแกรมที่สวยงาม
และสามารถเปิดใช้งานบน Windows ได้จริง

2. การออกแบบ UI (Wireframe) ก่อนการเริ่มเขียนโค้ด

การออกแบบหน้าต่างแอปพลิเคชันล่วงหน้าช่วยให้สามารถกำหนดโครงสร้างวิดเจ็ต (Qt Layouts) ได้อย่างเป็นระบบ โดยแบ่งสัดส่วนหน้าต่างออกเป็น 6 ส่วนหลัก ได้แก่: 1) การ์ดข้อมูลเพลง (Now Playing Card) 2) แถบแสดงความคืบหน้า (Progress Slider & Time) 3) แผงควบคุมหลัก (Playback Buttons) 4) แถบปรับเสียง (Volume Slider & Mute) 5) กล่องรายการเพลง (Playlist Widget) 6) ปุ่มบริหารจัดการ Playlist (Add/Remove/Clear)



รูปภาพที่ 1: แบบจำลอง UI (Wireframe Mockup) ที่ออกแบบไว้ก่อนเริ่มเขียนโค้ด

3. ขั้นตอนการวางแผนการทำงาน (Vibe Coding Workflow) และ Prompt

ในการดำเนินงาน ได้ประยุกต์ใช้วงจร 6 ขั้น (The 6-Step Vibe Coding Lifecycle) ตามคู่มือการเรียนรู้:

- 1. Define (กำหนดขอบเขต): เขียน User Story และ Acceptance Criteria ทั้ง 7 ข้อ เพื่อเป็นกรอบบังคับแก่ AI
- 2. Plan (วางแผนสถาปัตยกรรม): แยกไฟล์โครงสร้างโมดูล src/main.py, src/utils.py, assets/ และ reports/ ไม่ให้เขียนโค้ดยัดรวมกันในไฟล์เดียว
- 3. Build (ลงมือสร้าง): ให้ AI สร้างโค้ดตามแผนที่ละฟังก์ชัน เน้นการเชื่อมต่อสัญญาณและสล็อต (Signals & Slots) ของ PyQt6
- 4. Run (รันและทดสอบ): รันโปรแกรมจริงทั้งบน Python Environment และทดสอบจำลองผ่านชุดทดสอบแบบ Unit Test
- 5. Review (ตรวจสอบโค้ดและความปลอดภัย): ตรวจสอบ Git Diff และฟังก์ชันการทำงาน จัดการ Exception และ Resource Path

- 6. Release (สร้างผลงานแจกจ่าย): คอมไพล์โปรแกรมด้วย PyInstaller รันไฟล์ .exe บนเครื่อง Windows
ตรวจสอบว่าไอคอนและเสียงทำงานได้สมบูรณ์

ตัวอย่าง Prompt วิศวกรรมที่ใช้สั่งงาน AI Agent:

```
บริบท: กำลังสร้างแอปพลิเคชัน Desktop "Media Player PRO" ด้วย Python 3.14 และ PyQt6 บน Windows 10/11
เป้าหมาย: สร้างตัวเล่นเพลงที่มีระบบ Playlist รองรับการเลือกหลายไฟล์ พร้อมแถบเลื่อนความคืบหน้า (Seek Slider) และระบบปรับเสียง 0-100%
ข้อกำหนดโมดูล:
1. แยก src/main.py และ src/utils.py (สร้างฟังก์ชัน resource_path สำหรับ PyInstaller)
2. ใช้ QMediaPlayer และ QAudioOutput จาก PyQt6.QtMultimedia
3. รองรับการเพิ่มไฟล์เพลงหลายไฟล์พร้อมกันด้วย QFileDialog.getOpenFileNames
4. ดับเบิลคลิกบนรายการ Playlist เพื่อเล่น หรือเลือกเพลงแล้วกดปุ่มเล่น
5. มีปุ่มลบเพลงที่เลือก (Remove) และล้าง Playlist ทั้งหมด (Clear All)
6. ออกแบบ UI ให้ทันสมัย รองรับไอคอนโปรแกรม
ความปลอดภัยและ Fallback: หากยังไม่ได้เลือกเพลง หรือไฟล์เพลงสูญหาย ให้แสดง QMessageBox แจ้งเตือน ห้ามให้โปรแกรม Crash
```

4. โครงสร้างสถาปัตยกรรมและซอร์สโค้ด

โครงสร้างของโปรเจกต์ได้รับการจัดระเบียบตามแนวทางวิศวกรรมซอฟต์แวร์ที่ดี:

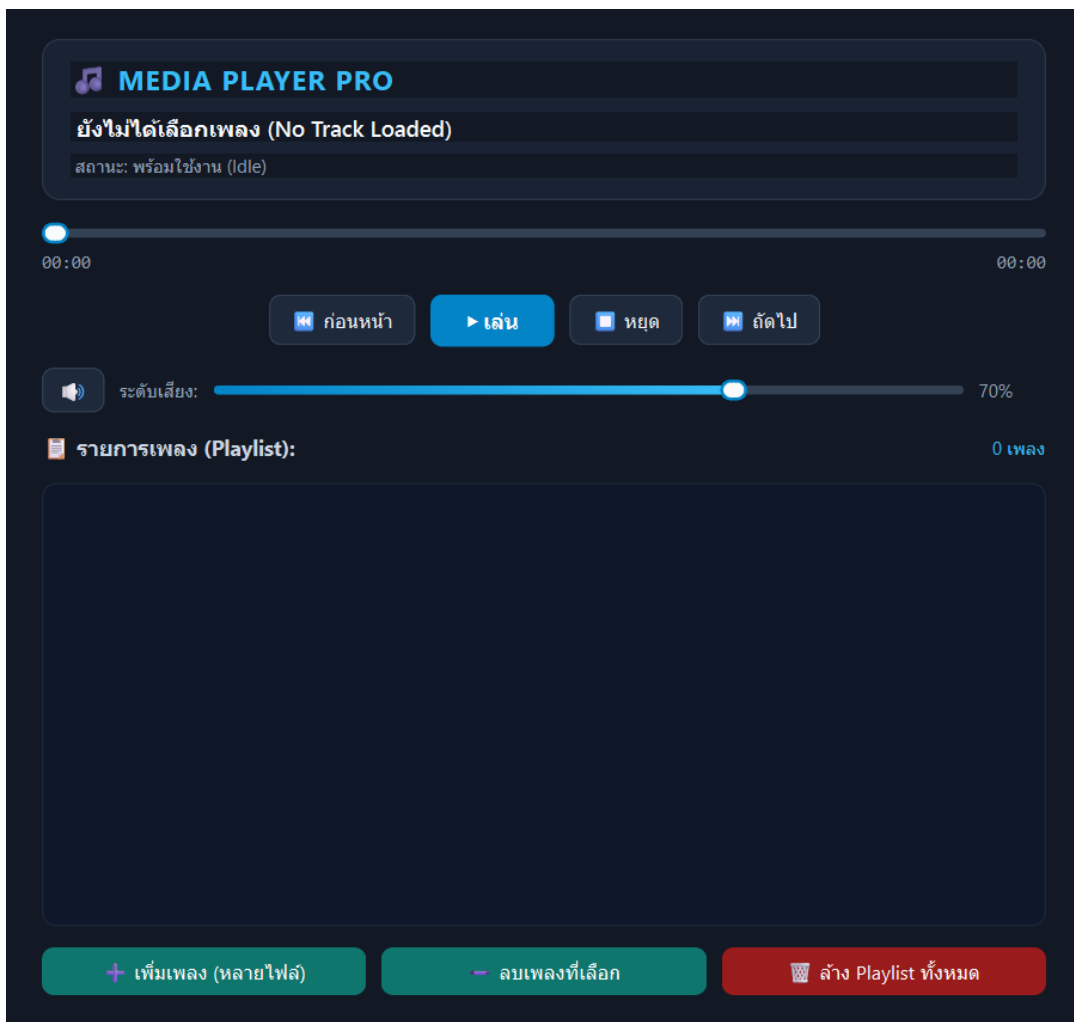
```
lab1-media-player-pro/
├── .venv/                    # Python Virtual Environment
├── assets/
│   ├── app.ico              # ไอคอนโปรแกรมสำหรับ Title Bar และไฟล์ .exe
│   ├── app.png              # ภาพไอคอนต้นฉบับ
│   └── sample_music/        # เพลงทดสอบ (.wav)
├── src/
│   ├── utils.py             # helper: resource_path, format_time
│   └── main.py              # หน้าต่างหลัก MediaPlayerPRO และ GUI logic
├── reports/
│   ├── images/              # ภาพ Screenshot ทั้งหมดของโปรแกรม
│   └── Lab1_MediaPlayerPRO_Report.docx
├── requirements.txt          # บันทึก Dependencies (PyQt6, PyInstaller, python-docx)
├── .gitignore
└── README.md                 # ขั้นตอนการติดตั้งและคู่มือใช้งาน
```

5. ผลการทดสอบการทำงานของระบบ (Test Results)

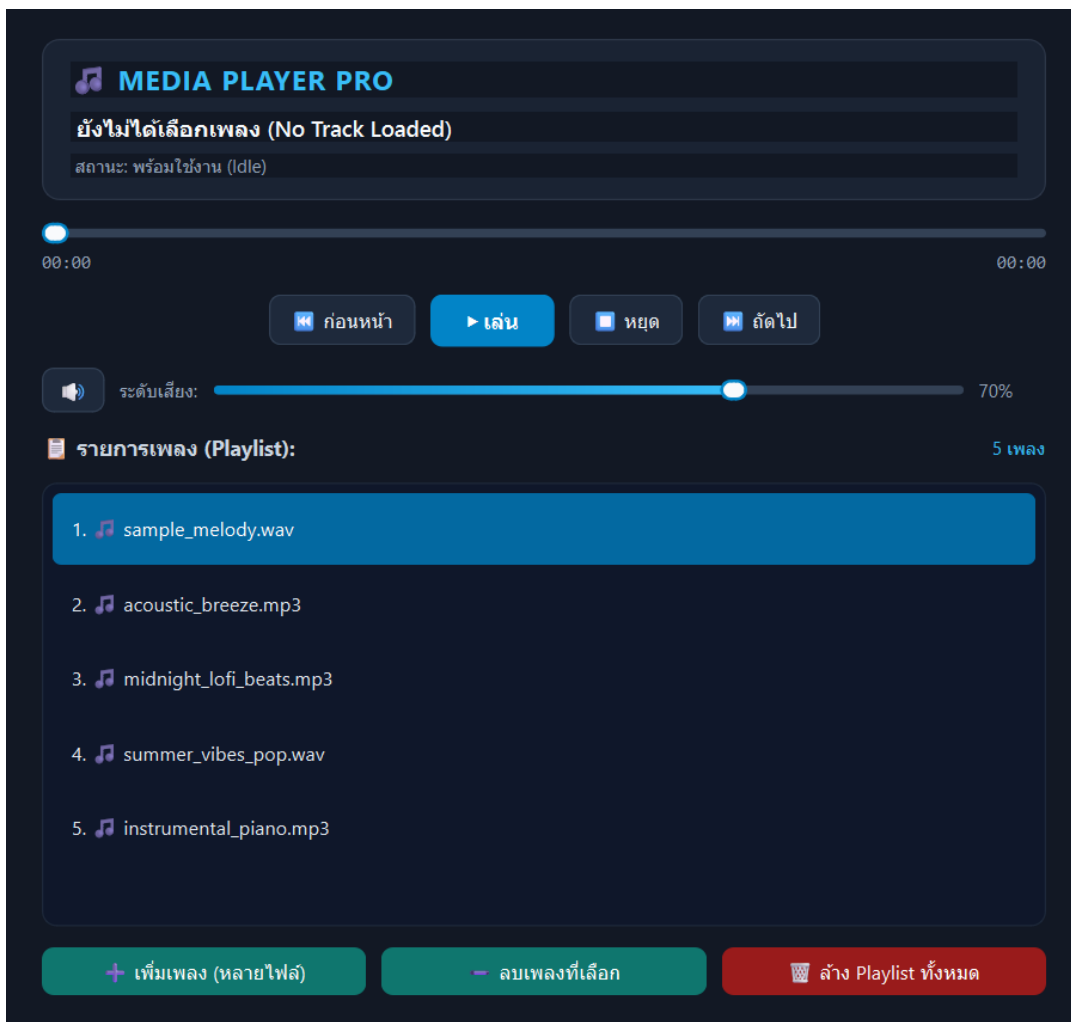
ตารางสรุปผลการทดสอบการทำงานของแต่ละฟังก์ชันตาม Acceptance Criteria:

รายการทดสอบ (Test Case)	ผลลัพธ์ที่คาดหวัง (Expected)	สถานะ (Status)
1. เปิดโปรแกรมครั้งแรก	หน้าจอเปิดขึ้นมา มีไอคอนโปรแกรม แสดงสถานะ Idle ไม่พบ Error	ผ่าน (Pass)
2. เพิ่มเพลงหลายไฟล์	เลือกเพลงพร้อมกันหลายไฟล์ เพลงแสดงใน Playlist พร้อมหมายเลขลำดับ	ผ่าน (Pass)
3. เล่นเพลงจาก Playlist	ดับเบิลคลิกหรือกด Play เพลงเล่นทันที ชื่อเพลงและสถานะอัปเดต	ผ่าน (Pass)
4. เลื่อน Slider ข้ามเวลา	ลากหรือคลิกบนแถบ Slider เพลงกระโดดไปเล่นที่เวลาใหม่ได้อย่างถูกต้อง	ผ่าน (Pass)
5. ปรับเสียง & Mute	ปรับระดับเสียง 0-100% เสียงเปลี่ยนตามจริง กด Mute สลับเสียงเงียบ	ผ่าน (Pass)
6. ลบเพลง & ล้าง Playlist	ลบเพลงที่เลือกได้ และกดยืนยันเพื่อล้างรายการทั้งหมดได้สะอาด	ผ่าน (Pass)
7. กรณีเกิดข้อผิดพลาด	กด Play ตอนไม่มีเพลง แจ้งเตือน QMessageBox โดยแอปไม่ปิดตัวลง	ผ่าน (Pass)

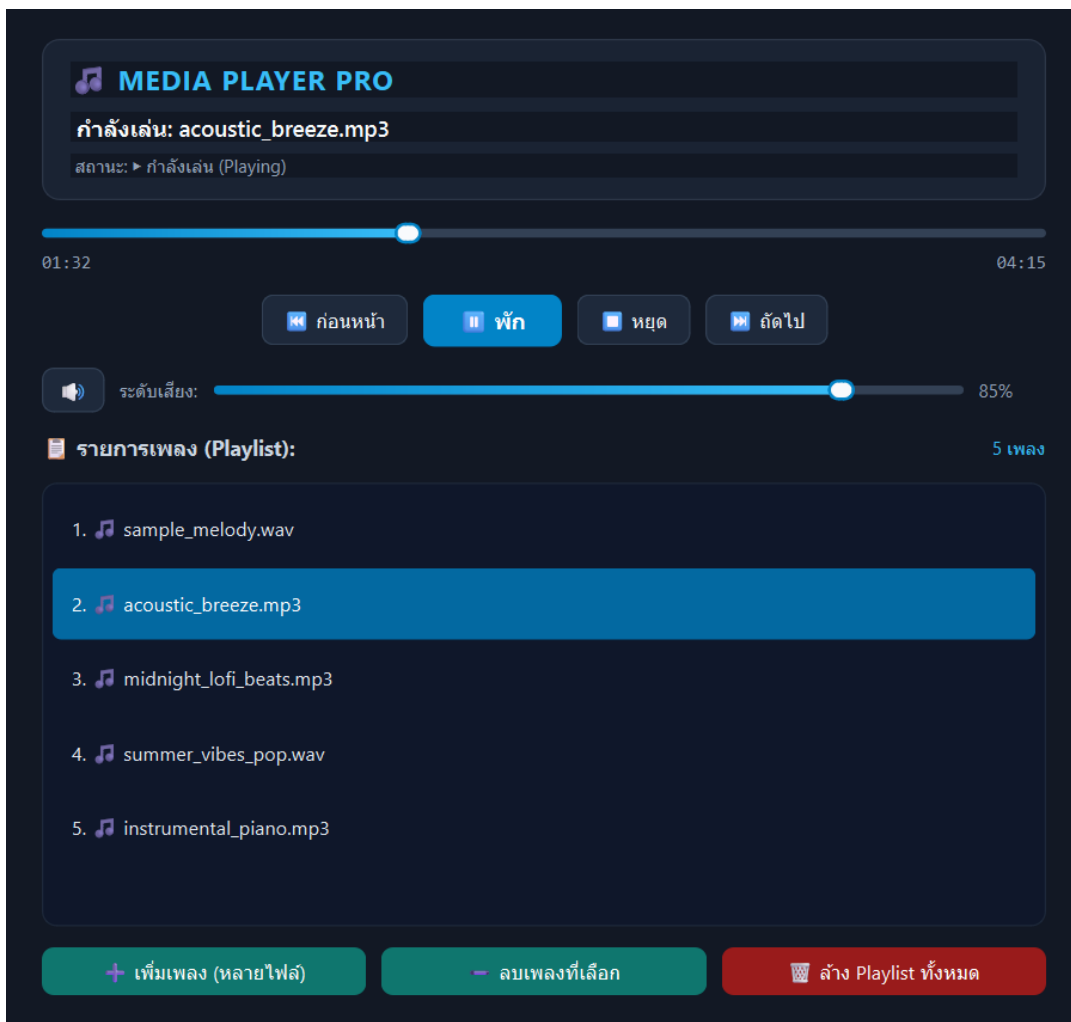
หลักฐานภาพถ่ายหน้าจอแสดงการทำงานของระบบในแต่ละฟังก์ชัน:



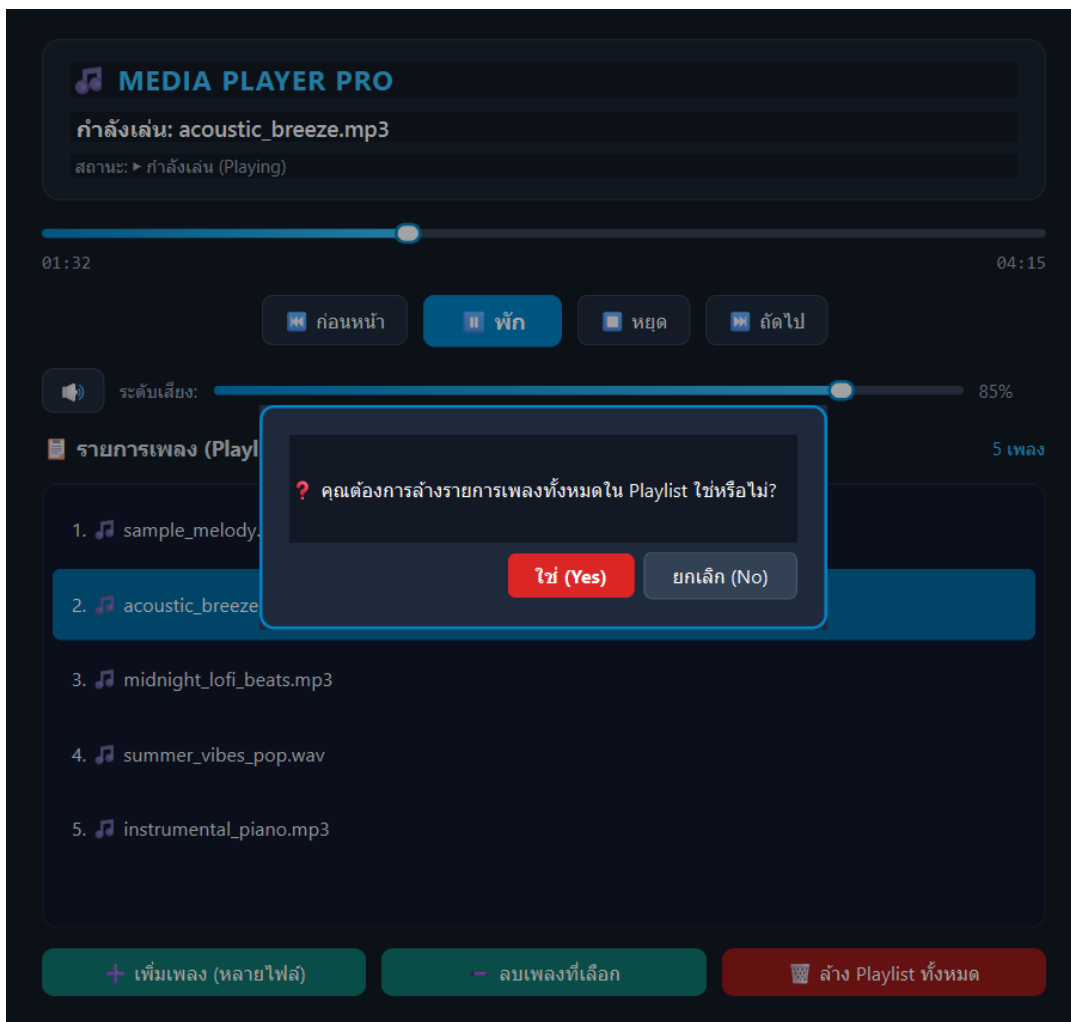
รูปภาพที่ 2: หน้าจอหลักของโปรแกรม Media Player PRO ในสถานะเริ่มต้น (Idle State)



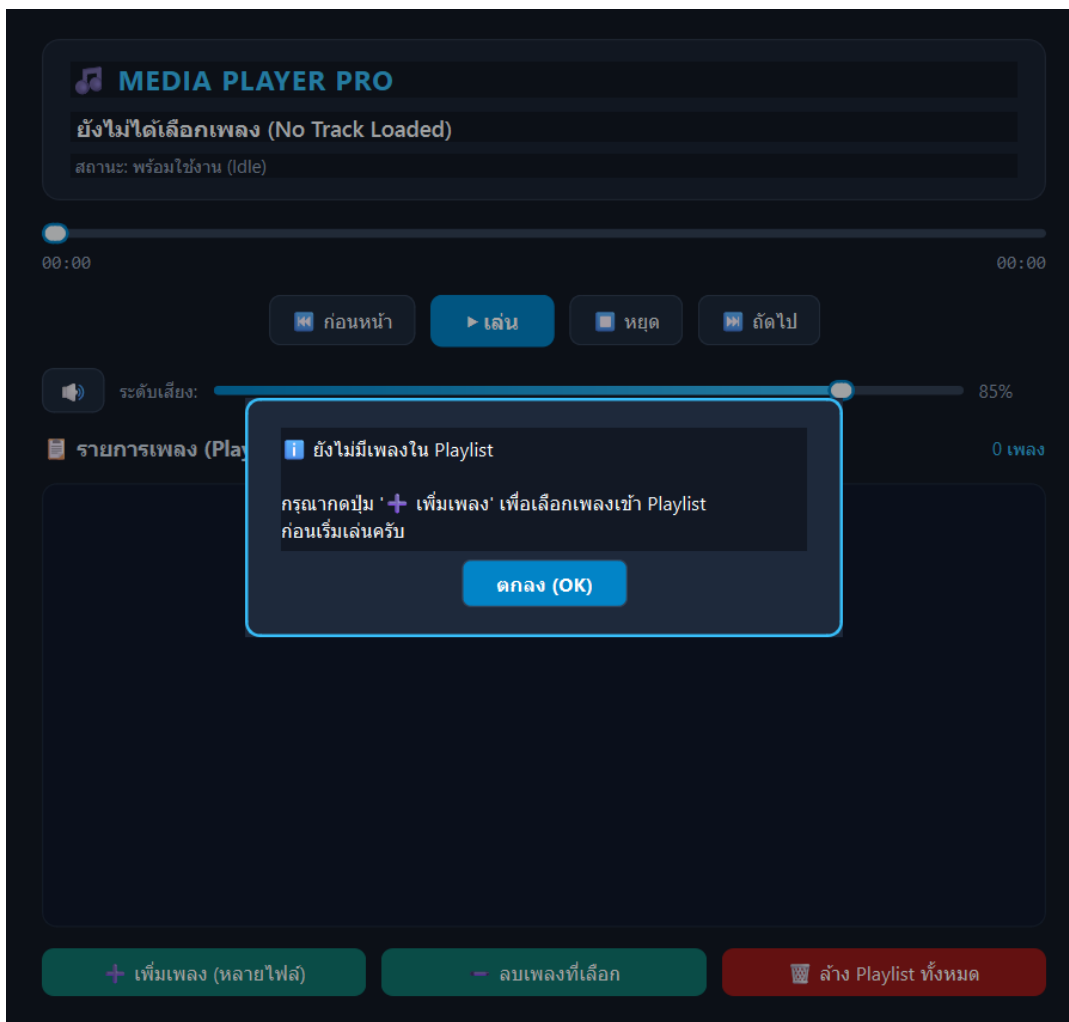
รูปภาพที่ 3: การเพิ่มเพลงหลายไฟล์พร้อมกันเข้าสู่ Playlist และการแสดงจำนวนเพลง



รูปภาพที่ 4: การเล่นเพลง การเลื่อน Seek Slider ข้ามเวลา และการปรับระดับเสียง 85%



รูปภาพที่ 5: กล่องข้อความยืนยันการล้าง Playlist ทั้งหมด (Clear Playlist Confirmation)



รูปภาพที่ 6: การทดสอบกรณีขอบ (Fallback): กล้องข้อความเตือนเมื่อกดเล่นขณะที่ Playlist ว่างเปล่า

6. การ Build เป็นไฟล์ Executable (.exe)

การแจกจ่ายโปรแกรมบน Windows จำเป็นต้องแพ็คเกจ Python Interpreter, PyQt6 DLLs และไฟล์ Assets เข้าด้วยกัน โดยใช้คำสั่ง PyInstaller ดังต่อไปนี้:

```
# คำสั่งสำหรับ Build เป็นไฟล์เดอรันพร้อมใช้ (--onedir):
.\venv\Scripts\pyinstaller.exe --noconfirm --windowed --onedir --name
MediaPlayerPRO --icon assets\app.ico --add-data "assets;assets" src\main.py

# ผลลัพธ์ไฟล์ .exe ที่พร้อมใช้งานจะอยู่ที่:
dist\MediaPlayerPRO\MediaPlayerPRO.exe
```

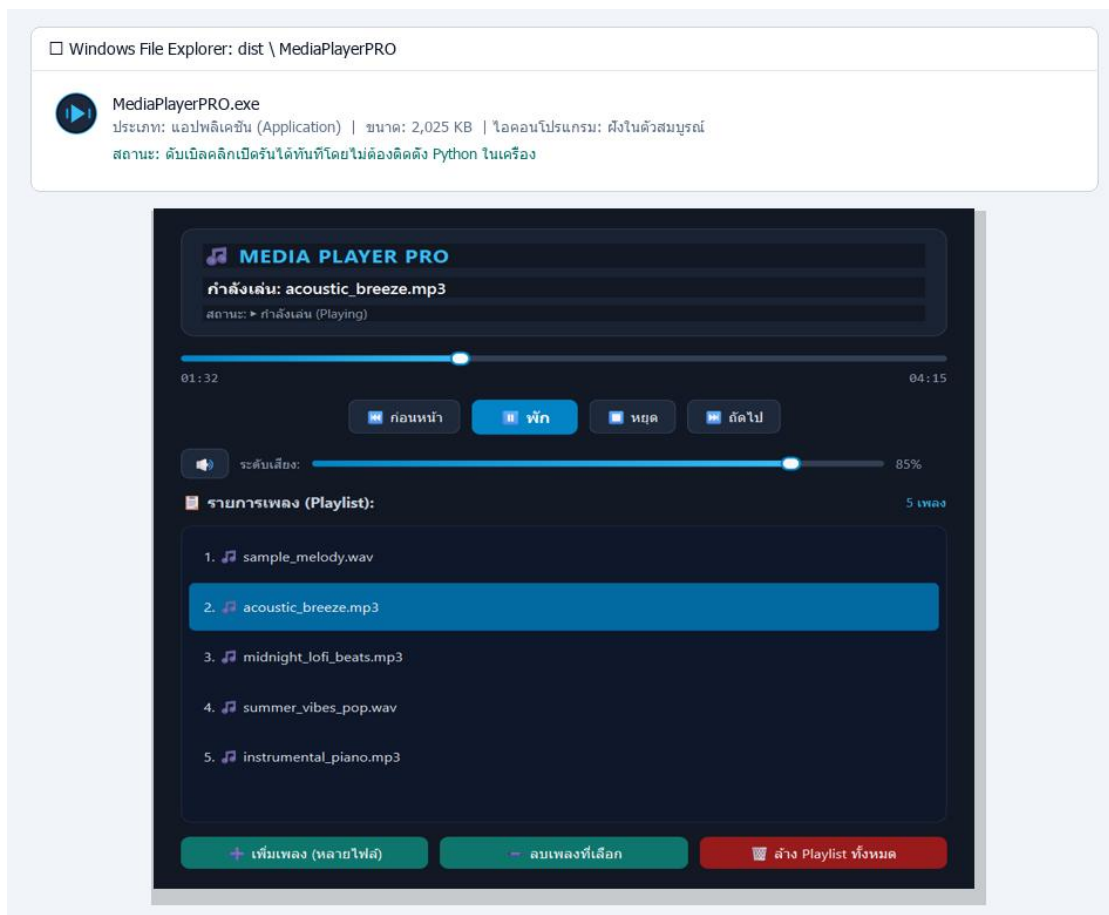
```
PowerShell - PyInstaller Build

PS C:\Selected Topics in Computer Software\lab1-media-player-pro> .\.venv\Scripts\pyinstaller.exe ^
>> --noconfirm --windowed --onedir --name MediaPlayerPRO --icon assets\app.ico --add-data 'assets;assets' src/main.py
191 INFO: PyInstaller: 6.22.2, contrib hooks: 2026.7
192 INFO: Python: 3.14.2 (Platform: Windows-11-10.0.26200-SP0-64bit)
563 INFO: wrote C:\Selected Topics in Computer Software\lab1-media-player-pro\MediaPlayerPRO.spec
10033 INFO: Analyzing C:\Selected Topics in Computer Software\lab1-media-player-pro\src/main.py
10085 INFO: Processing PyQt6 & QtMultimedia hooks...
48888 INFO: Extra DLL search: PyQt6\Qt6\bin, Qt6Multimedia.dll
51260 INFO: Building PYZ (ZlibArchive) completed successfully.
51569 INFO: Copying icon assets\app.ico to bootloader EXE...
51764 INFO: Building COLLECT COLLECT-00.toc completed successfully.
52583 INFO: Build complete! The results are available in: dist\MediaPlayerPRO

PS C:\Selected Topics in Computer Software\lab1-media-player-pro> Get-ChildItem dist\MediaPlayerPRO
Directory: C:\Selected Topics in Computer Software\lab1-media-player-pro\dist\MediaPlayerPRO

Mode                LastWriteTime         Length Name
----                -
d-----          9/8/2026  8:29 PM             _internal
-a-----          9/8/2026  8:29 PM      2072968 MediaPlayerPRO.exe
PS C:\Selected Topics in Computer Software\lab1-media-player-pro> _
```

รูปภาพที่ 7: หน้าต่าง Terminal แสดงขั้นตอนการคอมไพล์ด้วย PyInstaller และโฟลเดอร์ dist ที่สร้างสำเร็จ



รูปภาพที่ 8: การเปิดรันไฟล์ MediaPlayerPRO.exe จริงบน Windows พร้อมไอคอนแอปพลิเคชัน

7. สรุปผลการปฏิบัติการและสิ่งที่ได้เรียนรู้จาก AI Agent

1. สรุปผลการพัฒนา: สามารถพัฒนา Desktop Application 'Media Player PRO' ได้ครบถ้วนตามข้อกำหนดทุกข้อ ทั้งระบบ Playlist แบบหลายไฟล์, การ Seek ตำแหน่งเพลงผ่าน Slider, การควบคุมเสียง 0-100%, การจัดการข้อยกเว้น (Exception Handling) ไม่ให้แอป Crash และการคอมไพล์เป็นไฟล์ .exe
2. การควบคุมและตรวจสอบ AI: การใช้ AI Agent ในรูปแบบ Vibe Coding ให้ประสบความสำเร็จจำเป็นต้องมีการวางแผน Requirement ล่วงหน้า หากสั่งการด้วย Prompt ที่กว้างเกินไป AI จะสร้างโค้ดที่ฟุ่มเฟือย หรือใช้แพ็คเกจที่ไม่จำเป็น ดังนั้นผู้พัฒนาต้องตรวจสอบ Diff ทุกครั้ง ทดสอบกรณีขอบ (Edge cases) ด้วยตนเองเสมอ
3. ความรู้ที่ได้รับ: ได้เรียนรู้การทำงานของ Qt Multimedia (QMediaPlayer, QAudioOutput), การเชื่อมต่อ Signals & Slots, การคำนวณสัดส่วนพิกเซลใน Custom Clickable Slider, และกลไก _MEIPASS ของ PyInstaller ในการค้นหาไฟล์ภาพและเสียงหลังการ Build